

딥러닝 파이토치 교과서 5-1, 5-2

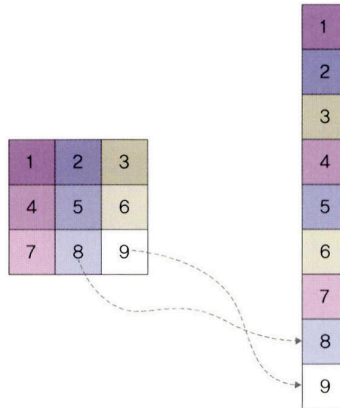
📄 제출	제출 전
📄 과제 유형	예습
📅 마감일	@2025년 3월 24일
☑ 발표	☐
☑ 완료	☐
☰ 주차	3 주차

5.1 합성곱 신경망

- 딥러닝의 역전파는 순전파 과정에 따라 계산된 오차 정보가 신경망의 모든 노드(출력층 → 은닉층 → 입력층)로 전송됨
 - 이는 복잡하고 많은 자원을 요구하며, 계산이 오래 걸림
 - **합성곱 신경망**으로 이를 해결하고자 함
- 합성곱 신경망은 이미지 전체를 한 번에 계산 x
- 이미지의 **국소적 부분**을 계산함으로써 시간과 자원을 절약해 이미지의 세밀한 부분까지 분석 가능한 신경망

5.1.1 합성곱층의 필요성

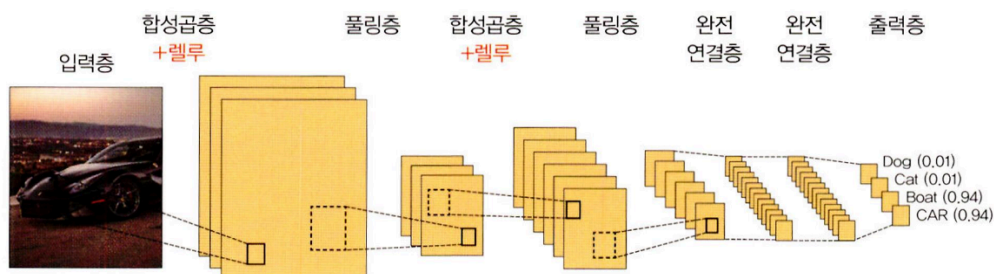
- 3 * 3 배열을 가지는 흑백 이미지는 은닉층으로 전달도리 때, 오른쪽 그림과 같이 펼쳐서 (flattening) 각 픽셀에 가중치를 곱하여 전달된다.
- 이렇게 이미지를 펼쳐서 전달하면, 데이터의 공간적 구조를 무시하게 되며, 이를 방지하기 위해 합성곱층이 나옴



- **합성곱 신경망:** 이미지와 영상을 처리하는데 유용

5.1.2 합성곱 신경망 (CNN) 구조

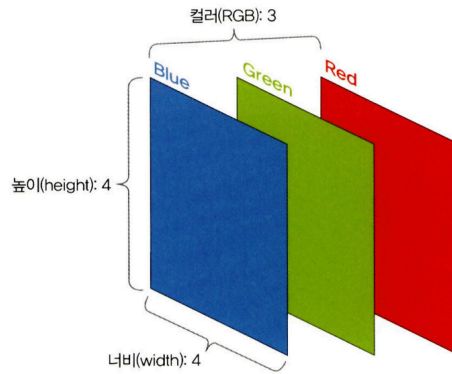
- **합성곱 신경망 (Convolutional Neural Network, CNN)**은 음성 인식이나 이미지/영상 인식에서 주로 사용됨.
- **다차원 배열 데이터**를 처리하도록 구성되어 컬러 이미지 같은 다차원 배열 처리에 특화되어 있으며, **입력층, 합성곱층, 풀링층, 완전 연결층, 출력층** 5개의 계층으로 구성되어 있음



- 합성곱층과 풀링층을 거치면서 입력 이미지의 주요 특성 벡터를 추출함.
- 추출된 feature vector들은 완전 연결층을 거치면서 **1차원 벡터**로 변환됨.
- 마지막으로 출력층에서 활성화 함수인 softmax 함수를 사용해 최종 결과가 출력됨

입력층

- 이미지 데이터가 최초로 거치게 되는 계층
- 이미지는 단순 1차원 데이터가 아닌 **높이, 너비, 채널**의 값을 가지는 **3차원 데이터**
 - 채널 = 1 when 이미지가 gray scale
 - 채널 = 3 when 이미지가 color (RGB)



- 높이 4, 너비 4, 채널 3 (컬러) → (4, 4, 3)

합성곱층 (Convolutional Layer)

- 입력 데이터에서 특성을 추출
- 입력 이미지가 들어왔을 때 이미지에 특성을 감지하기 위해 kernel이나 필터를 사용함
- 커널/필터는 이미지의 모든 영역을 훑으면서 특성을 추출하고, **feature map**을 만들
- 일반적으로, 커널은 3*3, 5*5 크기로 적용되며, 지정된 간격 (stride)에 따라 순차적으로 이동함.

• 그레이 스케일 이미지의 합성곱

- (6, 6, 1) 입력 이미지에 3*3 필터를 적용해보자. (스트라이드는 1이라고 가정)

1. 입력 이미지에 3*3 필터 적용

▼ 그림 5-4 입력 이미지에 3×3 필터 적용



- 입력 이미지와 필터를 포개놓고 대응되는 숫자끼리 곱한 후 모두 더한다
- $(1*1) + (0*0) + (0*1) + (0*0) + (1*1) + (0*0) + (0*1) + (0*0) + (1*1) = 3$

2. 필터가 1만큼 이동

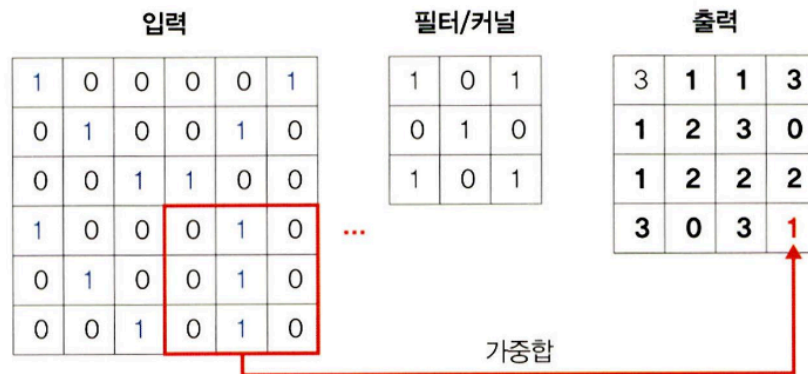
▼ 그림 5-5 입력 이미지에 필터가 1만큼 이동



- $(0*1) + (0*0) + (0*1) + (1*0) + (0*1) + (0*0) + (0*1) + (1*0) + (1*1) = 1$

3. 이 과정을 이미지 전체에 대해 반복

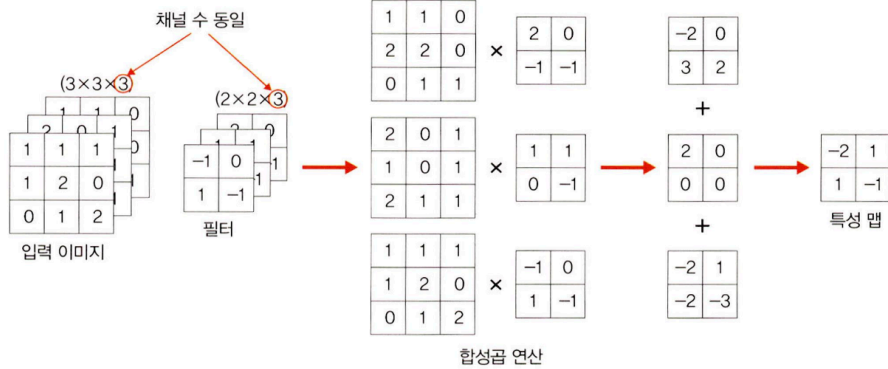
▼ 그림 5-9 입력 이미지에 필터가 1만큼 마지막으로 이동



4. 커널은 스트라이드 간격만큼 순회하면서 모든 입력 값과의 합성곱 연산으로 **새로운 특성 맵**을 만들게 되며, 원본 (6, 6, 1) 크기가 (4, 4, 1) 크기의 특성 맵으로 줄어들었음

• 컬러 이미지의 합성곱

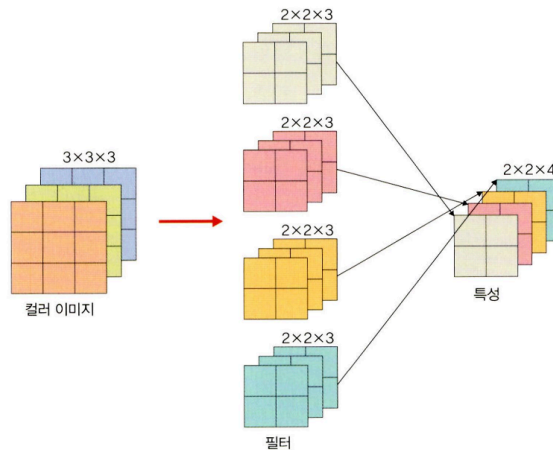
- 필터 채널이 3 (≠ 필터 개수가 3개, 필터 개수는 1개!!)
 - 이미지 채널 1개에 필터 채널 1개 적용
- RGB 각각에 서로 다른 가중치로 합성곱을 적용한 후 결과를 더해줌



- 입력 이미지의 R 채널과 필터의 R 가중치 행렬을 곱함
- G 채널과 필터의 G 가중치를 곱함
- B 채널과 필터의 B 가중치를 곱함
- 이 세 결과를 더해서 하나의 출력값을 만듦
- 출력값 = R합 + G합 + B합 = 스칼라 값 (하나의 숫자)

• 필터가 두 개 이상인 합성곱

▼ 그림 5-11 필터가 2 이상인 합성곱

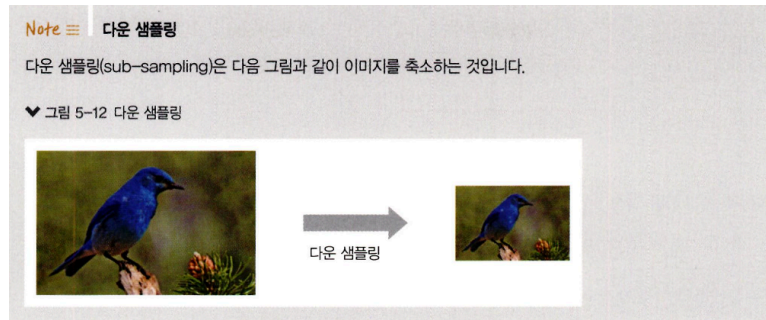


- 입력 데이터 : $W_1 * H_1 * D_1$ (W : 가로, H : 세로, D : 채널 또는 깊이)
- 하이퍼 파라미터: K : 필터 개수, F : 필터 크기, S : 스트라이드, P : 패딩
- 출력 데이터:
 - $W_2 = (W_1 - F + 2P)/S + 1$
 - $H_2 = (H_1 - F + 2P)/S + 1$

- $D_2 = K$

풀링층

- 합성곱층과 유사하게 특성 맵의 차원을 **다운 샘플링**하여 연산량을 감소시키고, 주요한 특성 벡터를 추출해 학습을 효과적으로 할 수 있게 함



- **최대 풀링(max pooling)**: 대상 영역에서 최댓값을 추출 (주로 사용)
- **평균 풀링(average pooling)**: 대상 영역에서 평균을 반환 (각 커널 값을 평균화 시켜 중요한 가중치를 갖는 값의 특성이 희미해질 수 있어서 잘 사용 x)
- 두 풀링의 계산 과정은 다르지만, **사용하는 파라미터는 동일함**
- **입력 데이터**: $W_1 * H_1 * D_1$ (W : 가로, H : 세로, D : 채널 또는 깊이)
- **하이퍼 파라미터**: F : 필터 크기, S : 스트라이드
- **출력 데이터**:
 - $W_2 = (W_1 - F) / S + 1$
 - $H_2 = (H_1 - F) / S + 1$
 - $D_2 = D_1$

- **최대 풀링 연산 과정** (스트라이드 = 2)

첫 번째 최대 풀링 과정

3, -1, -3, 1 값 중에서 최댓값(3)을 선택합니다.

▼ 그림 5-13 첫 번째 최대 풀링 과정



두 번째 최대 풀링 과정

12, -1, 0, 1 값 중에서 최댓값(12)을 선택합니다.

▼ 그림 5-14 두 번째 최대 풀링 과정



세 번째 최대 풀링 과정

2, -3, 3, -2 값 중에서 최댓값(3)을 선택합니다.

▼ 그림 5-15 세 번째 최대 풀링 과정



네 번째 최대 풀링 과정

0, 1, 4, -1 값 중에서 최댓값(4)을 선택합니다.

▼ 그림 5-16 네 번째 최대 풀링 과정



• 평균 풀링 연산 과정 (스트라이드 = 2)

- 각 필터의 평균을 계산

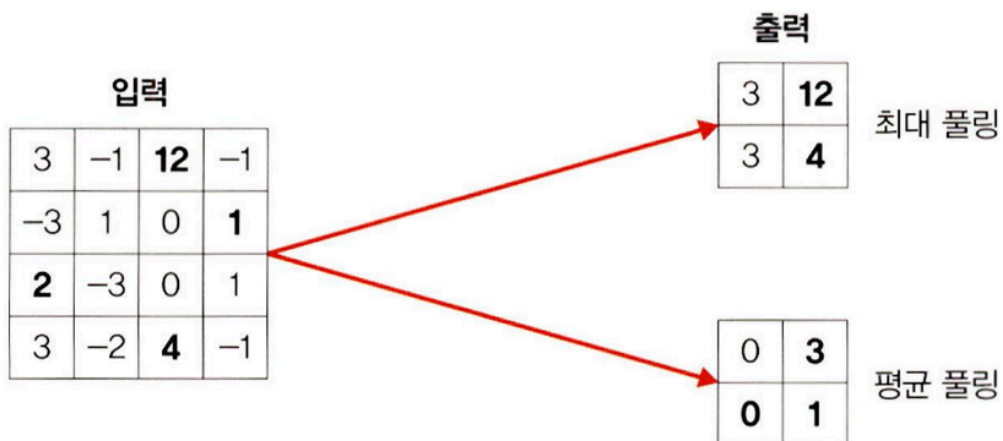
$$0 = (3 + (-1) + (-3) + 1) / 4$$

$$3 = (12 + (-1) + 0 + 1) / 4$$

$$0 = (2 + (-3) + 3 + (-2)) / 4$$

$$1 = (0 + 1 + 4 + (-1)) / 4$$

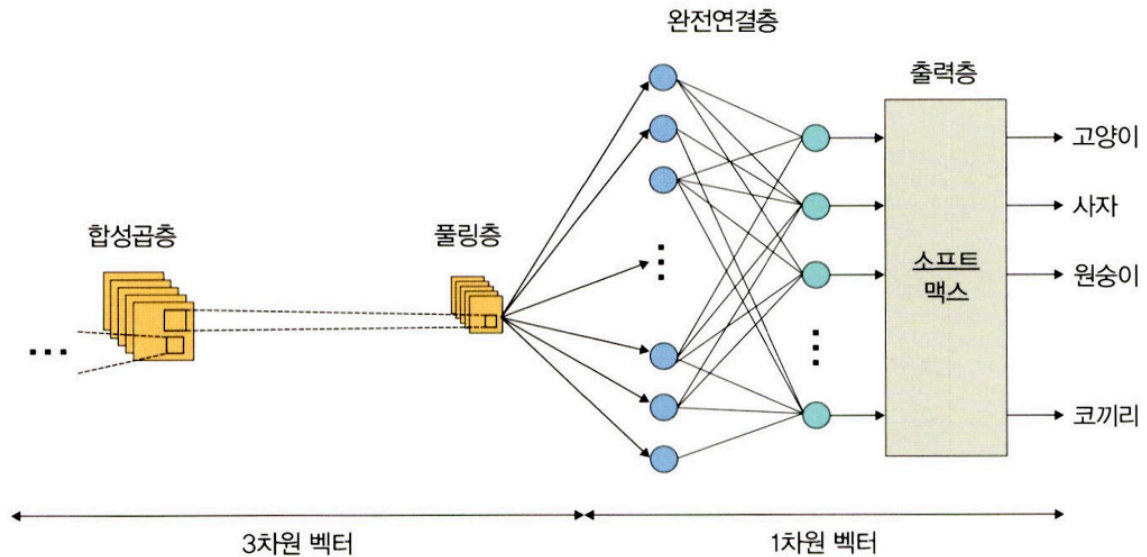
▼ 그림 5-17 최대 풀링과 평균 풀링 비교



완전 연결층

- 합성층과 풀링층을 거치면서 차원이 축소된 특성 맵은 최종적으로 완전 연결층으로 전달됨
- 이미지는 3차원 벡터에서 1차원 벡터로 펼쳐지게(flatten) 됨

♥ 그림 5-18 완전연결층



출력층

- Softmax 활성화 함수 사용해 입력받은 값을 0~1 사이의 확률값으로 출력
- 이미지가 각 label에 속할 확률값이 출력되며, 가장 높은 확률값을 갖는 레이블이 최종 값으로 선정됨

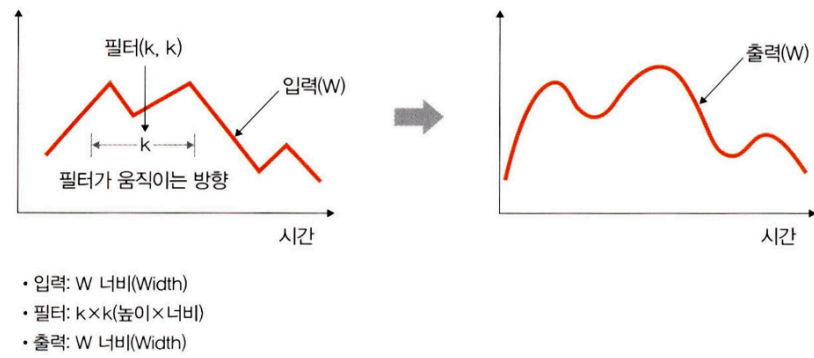
5.1.3 1D, 2D, 3D 합성곱

- 합성곱은 이동하는 방향의 수와 출력 형태에 따라 1D, 2D, 3D로 분류 가능

1D 합성곱

- 필터가 시간을 축으로 **좌우로만** 이동할 수 있는 합성곱
- 입력(W, 너비)과 필터(k)에 대한 출력은 W(너비), 1D의 배열이 됨
- 그래프의 곡선을 완화할 때 많이 사용
 - ex) 입력: [1, 1, 1, 1], 필터: [0.25, 0.5, 0.25]
 - 출력: $[1 \cdot 0.25 + 1 \cdot 0.5 + 1 \cdot 0.25, 1 \cdot 0.25 + 1 \cdot 0.5 + 1 \cdot 0.25, 1 \cdot 0.25 + 1 \cdot 0.5 + 1 \cdot 0.25]$
= [1, 1, 1]

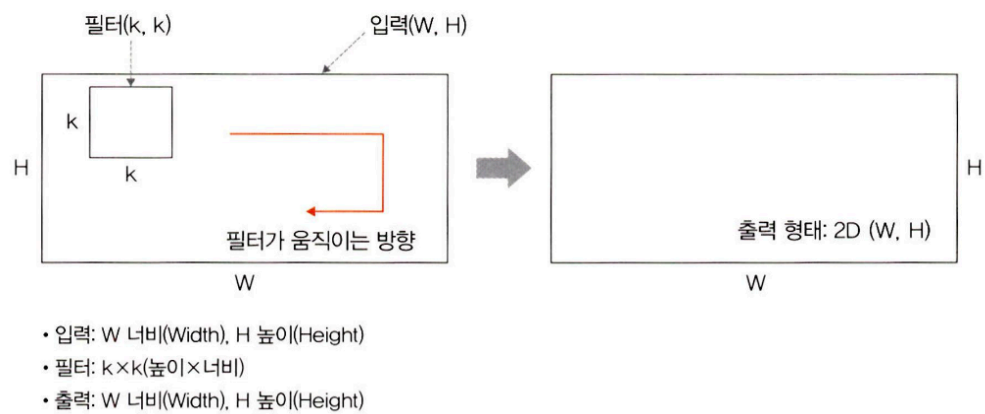
▼ 그림 5-19 1D 합성곱



2D 합성곱

- 필터가 방향 2 개로 움직이는 형태
- 입력(W, H)과 필터(k, k)에 대한 출력은 (W, H)가 되며, 출력 형태는 (W, H) 2D 행렬이 됨

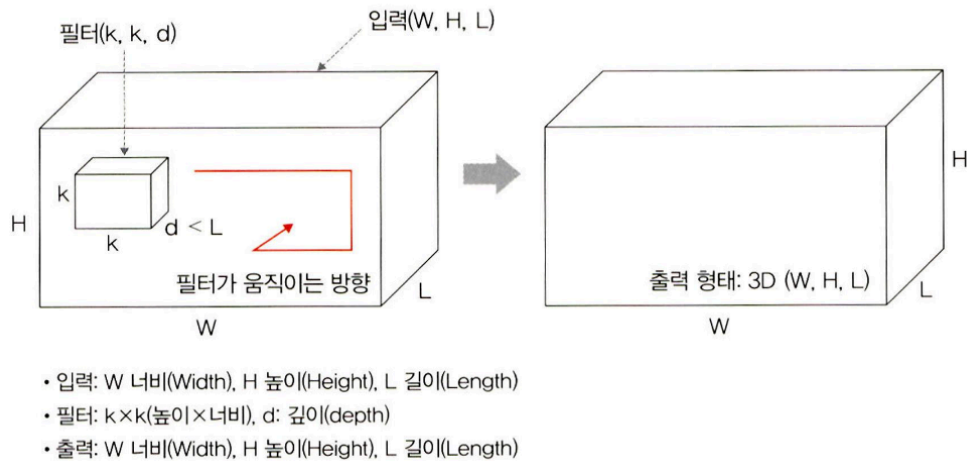
▼ 그림 5-20 2D 합성곱



3D 합성곱

- 필터가 세 개 방향으로 움직임
- 입력 (W, H, L)에 대해 필터 (k, k, d)를 적용하면 출력으로 (W, H, L) 3D 형태를 가짐 ($d < L$ 을 유지해야함)

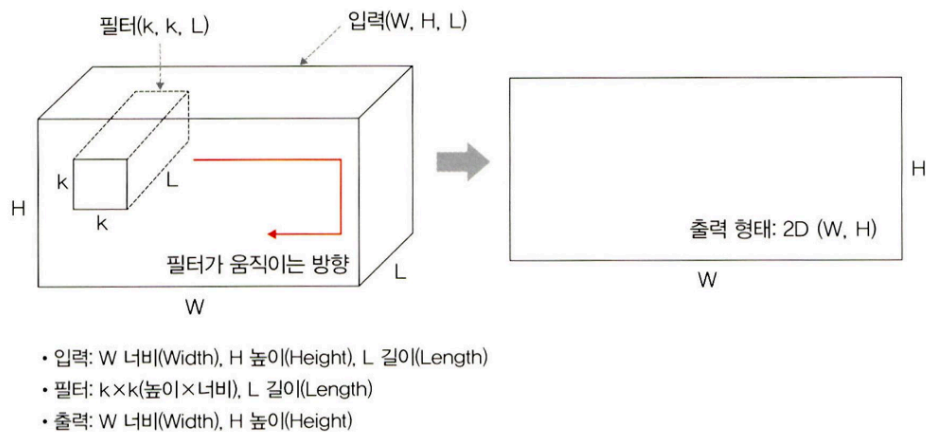
▼ 그림 5-21 3D 합성곱



3D 입력을 갖는 2D 합성곱

- 입력이 3D 형태임에도 출력 형태가 3D가 아닌 2D 행렬을 취하는 것
- 채널을 포함한 3차원 입력 (예: RGB 이미지)에 대해, 공간적으로 2D 합성곱을 수행하는 연산을 말합니다.
- 필터에 대한 길이(L)가 입력 채널의 길이(L)와 같기 때문에 만들어짐
 - 입력(W, H, L)에 필터 (k, k, L)를 적용하면 출력은 (W, H)가 됨

▼ 그림 5-22 3D 입력을 갖는 2D 합성곱



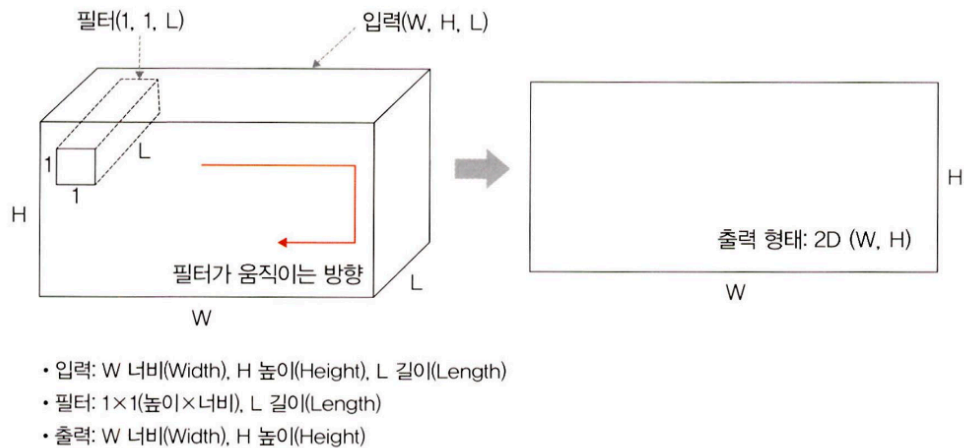
- ex) 입력: (32, 32, 3) → 너비 32, 높이 32, RGB 3채널 & 필터: (3, 3, 3) → 3x3 필터인데, 채널도 3개

1 * 1 합성곱

- 3D 형태로 입력되는 3D 입력을 갖는 2D 합성곱의 한 종류
- 커널의 공간 크기가 1x1로 한 픽셀 위치만 보지만, 채널 방향으로서는 전체를 다 봄

- 입력(W, H, L)에 필터 (1, 1, L)을 적용하면 출력은 (W, H)가 됨
- 1 * 1 합성곱에서 채널 수를 조절해서 연산량이 감소되는 효과가 있음

▼ 그림 5-23 1×1 합성곱



5.2 합성곱 신경망 맛보기

- `fashion_mnist` 데이터셋 사용
- `fashion_mnist` 데이터셋은 토치비전 (`torchvision`) 에 내장된 예제 데이터로 운동화, 셔츠, 샌들 같은 작은 이미지의 모음이며, 기본 MNIST 데이터셋처럼 열 가지로 분류될 수 있는 28x28 픽셀의 이미지 7 만 개로 구성되어 있다.
- 훈련 데이터 (`train_images`) 는 0 에서 255 사이의 값을 갖는 28X28 크기의 넘파이(Numpy) 배열이고, 레이블 (정답) 데이터 (`train_labels`) 는 0 에서 9 까지 정수 값을 갖는 배열, 0에서 9 까지 정수 값은 이미지 (운동화, 셔츠 등) 의 클래스를 나타내는 레이블이다.

```
0 : T-Shirt
1 : Trouser
2 : Pullover
3 : Dress
4 : Coat
5 : Sandal
6 : Shirt
7 : Sneaker
8 : Bag
9 : Ankle Boot
```

```
# 필요한 라이브러리 호출
import numpy as np
import matplotlib.pyplot as plt
```

```
import torch
import torch.nn as nn
from torch.autograd import Variable
import torch.nn.functional as F

import torchvision
import torchvision.transforms as transforms
from torch.utils.data import Dataset, DataLoader
```

```
# GPU 사용 권장, 설치되지 않았을 경우 CPU 사용
device = torch.device("mps" if torch.mps.is_available() else "cpu")
```

• 데이터셋 다운로드 받기

```
train_dataset = torchvision.datasets.FashionMNIST('data', download = True,
                                                  transform=transforms.Compose([transforms.ToT

test_dataset = torchvision.datasets.FashionMNIST('data', download = True, tra
                                                  transform=transforms.Compose([transforms.ToT
```

- `torchvision.datasets` 은 `torch.utils.data.Datasets` 의 하위 클래스로 다양한 데이터셋을 포함함
- `torchvision.datasets.FashionMNIST('data', download = True, train = False, transform=transforms.Compose([transforms.ToTensor()])))`
 - `data` : 데이터셋을 내려받을 위치
 - `download` : 첫번째 파라미터의 위치에 해당 데이터셋이 있는지 확인 후, 내려받음
 - `transform` : 이미지를 텐서(0~1)로 변경

```
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=100)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=100)
```

- 원하는 크기의 배치 단위로 데이터를 불러오거나, 순서가 무작위로 섞이도록할 수 있음
- Dataset에서 데이터 꺼내는 방법을 조절하는 장치!!!!

- Dataset: 어떤 데이터가 있는지 정의 (데이터셋의 내용)
- Dataloader: 그 데이터를 묶어서, 순서를 섞고, 학습에 쓸 수 있게 준비해주는 것

구분	Dataset	DataLoader
역할	데이터가 뭐냐? 를 정의함	**데이터를 어떻게 줄 거냐?**를 관리함
핵심기능	개별 샘플을 꺼낼 수 있게 함 (<code>__getitem__</code>)	배치 단위로, 섞어서, 반복적으로 데이터를 제공함
예시	"MNIST 데이터셋의 23번째 이미지는 5다"	"학습할 때 32개씩 랜덤하게 묶어서 주자"

• 분류에 사용될 클래스 정의

```
labels_map = {
    0 : 'T-Shirt',
    1 : 'Trouser',
    2 : 'Pullover',
    3 : 'Dress',
    4 : 'Coat',
    5 : 'Sandal',
    6 : 'Shirt',
    7 : 'Sneaker',
    8 : 'Bag',
    9 : 'Ankle Boot'
}
```

```
fig = plt.figure(figsize=(8, 8))
columns = 4
rows = 5
for i in range(1, columns * rows + 1): # 20개의 이미지를 무작위로 골라서 그림
    img_xy = np.random.randint(len(train_dataset)) # 0~train_dataset 길이 중 랜덤
    img = train_dataset[img_xy][0][0, :, :] # train_dataset을 이용한 3차원 배열 생성
    fig.add_subplot(rows, columns, i)
    plt.title(labels_map[train_dataset[img_xy][1]])
    plt.axis('off')
    plt.imshow(img, cmap='gray')
plt.show()
```

- `np.random.randint(10)` : 0~10의 임의의 숫자를 출력

- `np.random.randint(1, 10)` : 1~9의 임의의 숫자를 출력
- `np.random.rand(8)` : 0~1 사이의 정규표준분포 난수를 행렬로 (1*8)로 출력
- `np.random.rand(4,2)` : 0~1 사이의 정규표준분포 난수를 행렬로 (4*2)로 출력
- `np.random.randn(8)` : 평균이 0이고, 표준편차가 1인 가우시안 정규분포 난수를 (1*8) 행렬로 출력
- `np.random.randn(4,2)` : 평균이 0이고, 표준편차가 1인 가우시안 정규분포 난수를 행렬로 (4*2)로 출력

- `img = train_dataset[img_xy][0][0,:,:]` : train_dataset을 이용해 3차원 배열 생성
 - `train_dataset` 에서 `[img_xy][0][0, :, :]` 에 해당하는 요소값을 가져온다
 - `img_xy` : 0~train_dataset 길이만큼 중에서 무작위로 하나의 인덱스 선택 (랜덤한 이미지를 하나 뽑음)
 - `train_dataset[img_xy][0]` : label 제외 이미지 텐서만 가져오기
 - `train_dataset[img_xy][0][0,:,:]` : 첫 번째 채널을 가져옴



심층 신경망 생성

```
class FashionDNN(nn.Module):
    def __init__(self):
        super(FashionDNN, self).__init__()
        self.fc1 = nn.Linear(in_features=784, out_features=256)
        # in_features: 입력 크기
        # out_features: 출력 크기
        self.drop = nn.Dropout(0.25) # 전체 값 중 0.25의 확률로 0이 되고, 그렇지 않은
        self.fc2 = nn.Linear(in_features=256, out_features=128)
        self.fc3 = nn.Linear(in_features=128, out_features=10)

    def forward(self, input_data): # 모델이 학습 데이터를 입력 받아서 순전파 학습을 ?
        # nn.Linear의 첫 번째 파라미터를 입력받음
        out = input_data.view(-1, 784) # 텐서를 2차원 (?, 784)의 크기로 변경
```

```

out = F.relu(self.fc1(out)) # forward() 함수에서 정의
out = self.drop(out) # __init__ 함수에서 정의
out = F.relu(self.fc2(out))
out = self.fc3(out)
return out # nn.Linear의 두 번째 파라미터 출력

```

- `nn.Linear(in_features, out_features)` : 단순 선형 회귀 모델
 - `in_features` : 입력의 크기 (input size) → `forward` 로 전달
 - `out_features` : 출력의 크기 (output size) → `forward` 가 출력
- `torch.nn.Dropout(p)` : p만큼의 비율로 텐서의 값이 0이 되고, 0이 되지 않은 값들은 $(1/(1-p))$ 만큼 곱해져서 커짐
- `forward()` : 모델이 학습 데이터를 입력 받아서 순전파 학습을 진행
 - 객체를 데이터와 함께 호출하면 자동으로 실행됨
 - 활성화 함수에 입력된 x로부터 예측된 y를 얻음
- 활성화 함수 지정
 - `F.xx()` (`nn.functional.xx()`) 와 `nn.xx()` 차이 비교

구분	nn.xx	nn.functional.xx
형태	nn.Conv2d: 클래스 nn.Module 클래스를 상속받아 사용	nn.functional.conv2d: 함수 def function (input)으로 정의된 순수한 함수
호출 방법	먼저 하이퍼파라미터를 전달한 후 함수 호출을 통해 데이터 전달	함수를 호출할 때 하이퍼파라미터, 데이터 전달
위치	nn.Sequential 내에 위치	nn.Sequential에 위치할 수 없음
파라미터	파라미터를 새로 정의할 필요 없음	가중치를 수동으로 전달해야 할 때마다 자체 가중치를 정의

- `nn.xx()` 사용

```

import torch
import torch.nn as nn
inputs = torch.randn(64, 3, 244, 244)
conv = nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3, padding=1)
# 3 개의 채널이 입력되어 64 개의 채널이 출력되기 위한 연산 (3*3 크기의 커널 사용)
outputs = conv(inputs) # 함수 호출 후 데이터 전달
layer = nn.Conv2d(1, 1, 3)

```


- `nn.functional` 사용

```
import torch.nn.functional as F

inputs = torch.randn(64, 3, 244, 244) # 평균이 0이고 표준편차가 1인 정규분포
# 텐서의 크기가 (64, 3, 244, 244)
# 64: batch size (한 번에 64개 이미지 처리), 3: 채널수, (244, 244): (H, W)
weight = torch.randn(64, 3, 3, 3)
bias = torch.randn(64)
outputs = F.conv2d(inputs, weight, bias, padding = 1)
```

• 파라미터 정의

```
learning_rate = 0.001
model = FashionDNN()
model.to(device)
```

```
criterion = nn.CrossEntropyLoss() # 손실함수 정의
optimizer = torch.optim.Adam(model.parameters(), lr = learning_rate)
print(model)
```

```
FashionDNN(
  (fc1): Linear(in_features=784, out_features=256, bias=True)
  (drop): Dropout(p=0.25, inplace=False)
  (fc2): Linear(in_features=256, out_features=128, bias=True)
  (fc3): Linear(in_features=128, out_features=10, bias=True)
)
```

• 모델 학습

```
num_epochs = 5
count = 0
loss_list = []
iteration_list = []
accuracy_list = []

predictions_list = []
labels_list = []
```

```

for epoch in range(num_epochs):
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        train = Variable(images.view(100, 1, 28, 28))
        labels = Variable(labels)

        outputs = model(train) # 훈련 데이터로 모델 학습
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward() # 역전파 계산
        optimizer.step()
        count += 1

    if not (count % 50):
        total = 0
        correct = 0
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            labels_list.append(labels)
            test = Variable(images.view(100, 1, 28, 28))
            outputs = model(test)
            predictions = torch.max(outputs, 1)[1].to(device)
            predictions_list.append(predictions)
            correct += (predictions == labels).sum()
            total += len(labels)

        accuracy = correct * 100 / total
        loss_list.append(count)
        accuracy_list.append(accuracy)

    if not (count%500):
        print("Iteration: {}, Loss: {}, Accuracy: {}".format(count, loss.data, ac

```

Iteration: 500, Loss: 0.5958743095397949, Accuracy: 83.2300033569336%
 Iteration: 1000, Loss: 0.48090147972106934, Accuracy: 84.18000030517578%
 Iteration: 1500, Loss: 0.3547356426715851, Accuracy: 84.77999877929688%
 Iteration: 2000, Loss: 0.36035120487213135, Accuracy: 85.5%
 Iteration: 2500, Loss: 0.2728109061717987, Accuracy: 86.27999877929688%
 Iteration: 3000, Loss: 0.2886870503425598, Accuracy: 86.41999816894531%

- 86%의 높은 수치
- `images, labels = images.to(device), labels.to(device)` : 모델이 데이터를 처리하기 위해서는 모델과 데이터가 동일한 장치에 있어야함
- `Variable` : 자동 미분을 수행하는 파이토치의 핵심 패키지 Autograd에서 사용.
 - 자동 미분 값을 저장하기 위해 `tape`를 사용하여 순전파 단계에서 수행하는 모든 연산을 저장
 - 역전파 단계에서 저장된 값들을 꺼내서 사용 (역전파를 위한 미분값 자동 계산)
- 분류 문제의 정확도
 - `classification accuracy = correct predictions / total predictions * 100`
 - OR `error rate = (1 - (correct predictions / total predictions)) * 100`
 - 하지만, 클래스가 3개 이상일 때는
 - 모든 클래스가 동등하게 고려된 것인지 생각
 - 모든 데이터가 특정 클래스에 속할 때도 예측 결과가 높게 나오기 때문에 데이터 특성에 따라 정확도를 잘 관측해야함

합성곱 네트워크 생성

```
class FashionCNN(nn.Module):
    def __init__(self):
        super(FashionCNN, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        self.layer2 = nn.Sequential(
            nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3),
            nn.BatchNorm2d(64),
```

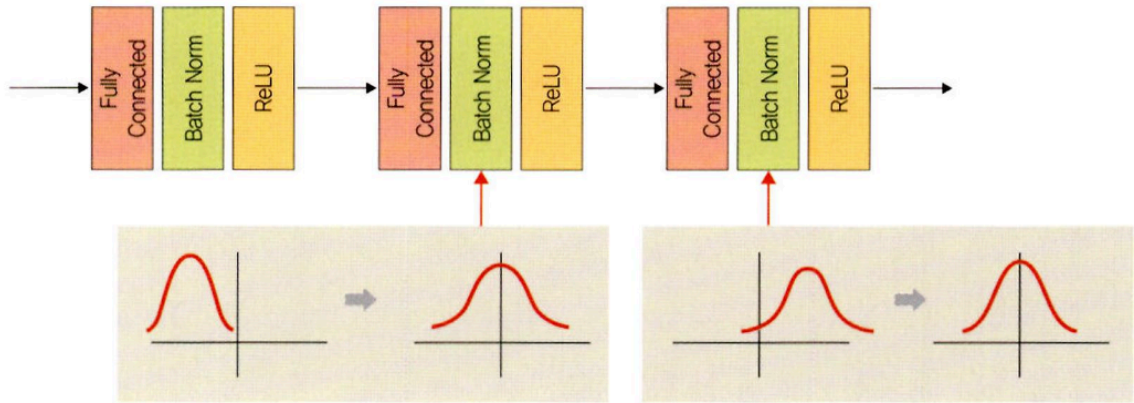
```

        nn.ReLU(),
        nn.MaxPool2d(2)
    )
    self.fc1 = nn.Linear(in_features=64 * 6 * 6, out_features=600)
    self.drop = nn.Dropout2d(0.25)
    self.fc2 = nn.Linear(in_features=600, out_features=120)
    self.fc3 = nn.Linear(in_features=120, out_features=10)

    def forward(self, x):
        out = self.layer1(x)
        out = self.layer2(out)
        out = out.view(out.size(0), -1)
        out = self.fc1(out)
        out = self.drop(out)
        out = self.fc2(out)
        out = self.fc3(out)
        return out

```

- `nn.Sequential` : `__init__()` 에서 사용할 네트워크 모델들을 정의해 줄 뿐만 아니라 , `forward()` 함수에서 구현될 순전파를 계층 (ayer) 형태로 좀 더 가독성이 뛰어난 코드로 작성할 수 있음
 - $Wx + b$ 와 같은 수식과 활성화 함수를 연결해 주는 역할
 - 여러 개의 계층을 하나의 컨테이너에 구현하는 방법
- `nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)` : 합성곱 연산을 통해 이미지의 특징을 추출함
 - `in_channels` : 입력 채널의 수 (흑백 이미지=1, RGB 이미지=3)
 - `out_channels` : 출력 채널의 수
 - `kernel_size` : 커널(필터)의 크기, 입력 데이터를 스트라이드 간격으로 순회하면서 합성곱 계산
 - 직사각형으로 연산하고 싶으면 `kernel_size = (3,5)` 로 설정
 - `padding` : 출력 크기를 조정하기 위해 입력 데이터 주위에 0을 채움. 패딩 값이 클수록 출력 크기도 커짐
- `nn.BatchNorm2d(32)` : 학습 과정에서 각 배치 단위별로 데이터가 다양한 분포를 가지더라도 평균과 분산을 이용해 정규화시켜 (0, 1)의 가우시안 형태로 만드는 것



- `nn.MaxPool2d(kernel_size=2, stride=2)` : 이미지 크기를 축소시키는 용도로 사용.
 - 풀링 계층은 합성곱층의 출력 데이터를 입력으로 받아서 출력 데이터의 크기를 줄이거나 특정 데이터를 강조하는 용도로 사용
 - `kernel_size` : m X n 행렬로 구성된 가중치
 - `stride` : 입력 데이터에 커널을 적용할 때 이동할 간격을 의미, 스트라이드 값이 커지면 출력 크기는 작아진다.
- `self.fc1 = nn.Linear(in_features=64 * 6 * 6, out_features=600)`
 - 클래스를 분류하기 위해서는 이미지 형태의 데이터를 배열 형태로 변환하여 작업해야 하는데 Conv2d에서 사용하는 하이퍼 파라미터 값들에 따라 출력 크기가 달라짐.
 - 줄어든 출력 크기가 완전 연결층으로 전달됨
 - `in_features` : 입력 데이터(이전까지의 Conv2d, MaxPool2d는 이미지 데이터를 입력으로 받아서 처리)를 1차원으로 변경
 - `out_features` : 출력 데이터의 크기

Conv2d 계층에서의 출력 크기 구하는 공식

- 출력 크기 = $(W - F + 2P) / S + 1$
 - W : 입력 데이터의 크기(input_volume_size)
 - F : 커널 크기(kernel_size)
 - P : 패딩 크기(padding_size)
 - S : 스트라이드(strides)

예를 들어 첫 번째 Conv2d 계층은 다음과 같습니다.

```
nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)
```

따라서 출력 크기는 다음과 같이 계산할 수 있습니다.

$$(784 - 3 + (2 * 1)) / 1 + 1 = 784$$

(fashion_mnist의 입력 데이터 크기는 784이며, stride가 명시되어 있지 않다면 stride가 본값은 (1,1)입니다)

계산 결과를 적용하면 출력의 형태는 [32, 784, 784]가 됩니다.

MaxPool2d 계층에서의 출력 크기 구하는 공식

- 출력 크기 = IF/F
 - IF : 입력 필터의 크기(input_filter_size, 또한 바로 앞의 Conv2d의 출력 크기이기도 합니다)
 - F : 커널 크기(kernel_size)

예를 들어 첫 번째 MaxPool2d 계층은 다음과 같습니다.

```
nn.MaxPool2d(kernel_size=2, stride=2)
```

따라서 출력 크기는 다음과 같이 계산할 수 있습니다.

$$784 / 2 = 392$$

(784는 첫 번째 Conv2d에서 계산한 결과입니다)

계산 결과를 적용하면 출력의 형태는 [32, 392, 392]가 됩니다. 그리고 가장 앞의 32는 바로 앞 Conv2d 계층의 out_channels입니다.

• 합성곱 네트워크를 위한 파라미터 정의

```
learning_rate = 0.001
model = FashionCNN()
model.to(device)
```

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr = learning_rate)
print(model)
```

```

FashionCNN(
  (layer1): Sequential(
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (layer2): Sequential(
    (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1))
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (fc1): Linear(in_features=2304, out_features=600, bias=True)
  (drop): Dropout2d(p=0.25, inplace=False)
  (fc2): Linear(in_features=600, out_features=120, bias=True)
  (fc3): Linear(in_features=120, out_features=10, bias=True)
)

```

- 모델 학습

```

num_epochs = 5
count = 0
loss_list = []
iteration_list = []
accuracy_list = []

predictions_list = []
labels_list = []

for epoch in range(num_epochs):
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        train = Variable(images.view(100, 1, 28, 28))
        labels = Variable(labels)

        outputs = model(train)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        count += 1

    if not (count % 50):
        total = 0

```

```

correct = 0
for images, labels in test_loader:
    images, labels = images.to(device), labels.to(device)
    labels_list.append(labels)
    test = Variable(images.view(100, 1, 28, 28))
    outputs = model(test)
    predictions = torch.max(outputs, 1)[1].to(device)
    predictions_list.append(predictions)
    correct += (predictions == labels).sum()
    total += len(labels)

accuracy = correct * 100 / total
loss_list.append(loss.data)
iteration_list.append(count)
accuracy_list.append(accuracy)

if not (count % 500):
    print("Iteration: {}, Loss: {}, Accuracy: {}%".format(count, loss.data, ac

```

```

/opt/anaconda3/envs/euron/lib/python3.9/site-packages/torch/nn/functional.py:1538: UserWarning: dropout2d: Receiv
warnings.warn(warn_msg)
Iteration: 500, Loss: 0.4630039930343628, Accuracy: 87.37999725341797%
Iteration: 1000, Loss: 0.30371615290641785, Accuracy: 87.44999694824219%
Iteration: 1500, Loss: 0.3135014772415161, Accuracy: 87.95999908447266%
Iteration: 2000, Loss: 0.22828266024589539, Accuracy: 89.52999877929688%
Iteration: 2500, Loss: 0.13720618188381195, Accuracy: 89.66999816894531%
Iteration: 3000, Loss: 0.15184128284454346, Accuracy: 90.22000122070312%

```

심층 신경망과 비교하여 정확도가 약간 더 높음 (약 4%). 심층 신경망과 큰 차이는 없지만, 이미지 데이터가 많아지면 단순 심층 신경망으로는 정확한 특성 추출 및 분류가 불가능하므로 학습 신경망을 생성하는 연습이 필요